# SQL Core Cheat Sheet

## 1. JOINS

Joins combine rows from two or more tables based on related columns.

### INNER JOIN

Returns only matching rows from both tables.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

**Venn Diagram:** Intersection of two circles (matching records only)

---

### LEFT OUTER JOIN (LEFT JOIN)

Returns all rows from left table + matching rows from right table. Non-matching right rows show NULL.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

**Result:** All employees, even if no matching department.

---

### RIGHT OUTER JOIN (RIGHT JOIN)

Returns all rows from right table + matching rows from left table. Non-matching left rows show NULL.

```
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

**Result:** All departments, even if no matching employees.

---

### FULL OUTER JOIN

Returns all rows from both tables. Non-matching rows show NULL.

```
SELECT e.name, d.department_name
FROM employees e
```

```
FULL OUTER JOIN departments d
ON e.dept_id = d.dept_id;
```

**Note:** Not supported in MySQL; use UNION as workaround.

---

# 2. SUBQUERIES

A subquery (inner query) is a query within another query. Used in SELECT, WHERE, FROM, or HAVING clauses.

## Single-Row Subquery

Returns exactly one row. Use comparison operators: `=`, `>`, `<`, `>=`, `<=`, `<>`

```
SELECT * FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

**Use Case:** Find employees with maximum salary.

---

## Multi-Row Subquery

Returns multiple rows. Use operators: `IN`, `ANY`, `ALL`

```
SELECT * FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'New York');
```

**Use Case:** Find all employees in specific departments.

---

## Correlated Subquery

References columns from outer query. Executes once per row.

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e1.dept_id);
```

**Use Case:** Find employees earning more than their department average.

---

## Subquery in FROM Clause

Creates derived table.

```
SELECT * FROM (
SELECT name, salary FROM employees WHERE salary > 50000
) high_earners;
```

---

# 3. WINDOW FUNCTIONS

Perform calculations across a set of rows without collapsing results (unlike aggregate functions).

### General Syntax

window_function() OVER (PARTITION BY column ORDER BY column [ASC|DESC])

---

### ROW_NUMBER()

Assigns unique sequential number to each row. Handles ties by assigning next number.

```
SELECT name, salary,
ROW_NUMBER() OVER (ORDER BY salary DESC) as row_num
FROM employees;
```

**Output:**

| name | salary | row_num |
|---|---|---|
| Alice | 80000 | 1 |
| Bob | 75000 | 2 |
| Carol | 75000 | 3 |

**Note:** Carol gets 3 despite same salary as Bob (no gaps, always unique).

---

### RANK()

Assigns rank to rows. Ties get same rank, next rank skips numbers.

```
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) as rank
FROM employees;
```

**Output:**

| name | salary | rank |
|---|---|---|
| Alice | 80000 | 1 |
| Bob | 75000 | 2 |
| Carol | 75000 | 2 |
| David | 70000 | 4 |

**Note:** Two employees at rank 2 → rank 4 is next (skips rank 3).

---

### DENSE_RANK()

Assigns rank without gaps. Ties get same rank, next rank continues consecutively.

```
SELECT name, salary,
DENSE_RANK() OVER (ORDER BY salary DESC) as dense_rank
FROM employees;
```

**Output:**

| name | salary | dense_rank |
|---|---|---|
| Alice | 80000 | 1 |
| Bob | 75000 | 2 |
| Carol | 75000 | 2 |
| David | 70000 | 3 |

**Note:** Rank 3 follows rank 2 directly (no gaps).

---

### ROW_NUMBER vs RANK vs DENSE_RANK

| Function | Unique | Ties Handled | Gaps | Best For |
|---|---|---|---|---|
| ROW_NUMBER | ✓ Yes | Skip to next | No | Unique row IDs |
| RANK | ✗ No | Same rank, skip | Yes | Competition ranking |
| DENSE_RANK | ✗ No | Same rank, no skip | No | Pagination queries |

---

### Partition By Clause

Divides rows into partitions before applying window function.

```
SELECT name, dept_id, salary,
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) as rank_in_dept
FROM employees;
```

**Result:** Ranking resets for each department.

---

# 4. DATABASE NORMALIZATION (1NF - BCNF)

Organizing tables to minimize redundancy and ensure data integrity.

### Normal Forms Summary

| Form | Goal | Rule | Problem Fixed |
|---|---|---|---|
| **1NF** | Atomic Values | No repeating groups | Redundant data |
| **2NF** | Full Dependency | Remove partial dependencies | Composite key anomalies |
| **3NF** | No Transitive | Non-keys don't depend on non-keys | Chain dependencies |
| **BCNF** | Superkey Focus | Every determinant is a candidate key | Complex candidate keys |

---

### 1NF: First Normal Form

**Rule:** Every field contains atomic (single, indivisible) values. No repeating groups.

**✗ Not 1NF**
StudentCourses
StudentID | Name | Courses
101 | Alice | Math, Physics, Chemistry

✓ **1NF**
StudentCourses
StudentID | Name | Course
101 | Alice | Math
101 | Alice | Physics
101 | Alice | Chemistry

---

### 2NF: Second Normal Form

**Rule:** Must be 1NF + no partial dependencies. Non-key attributes depend on entire primary key (not part of it).

**✗ Not 2NF** (Composite PK: StudentID + CourseID)
StudentCourses
StudentID | CourseID | CourseName | Instructor
101 | C1 | Math | Dr. Smith
Issue: CourseName depends only on CourseID (partial)

✅ **2NF** (Separate tables)
StudentCourses Courses
StudentID | CourseID CourseID | CourseName | Instructor
101 | C1 C1 | Math | Dr. Smith

---

### 3NF: Third Normal Form

**Rule:** Must be 2NF + no transitive dependencies. Non-key attributes depend only on primary key, not on other non-key attributes.

❌ **Not 3NF**
Orders
OrderID | CustomerID | CustomerName | City
1001 | C1 | Alice | New York
Issue: CustomerName and City depend on CustomerID (transitive)

✅ **3NF** (Separate tables)
Orders Customers
OrderID | CustomerID | CustomerID | CustomerName | City
1001 | C1 | C1 | Alice | New York

---

### BCNF: Boyce-Codd Normal Form

**Rule:** Stricter than 3NF. Every determinant (attribute that determines another) must be a superkey (candidate key).

❌ **Not BCNF**
Professor_Course
ProfessorID | CourseID | Time
P1 | C1 | 9:00 AM
Issue: CourseID determines Time, but CourseID is not a superkey

✅ **BCNF** (Separate tables)
Professor_Course Course_Time
ProfessorID | CourseID | CourseID | Time
P1 | C1 | C1 | 9:00 AM

---

## Quick Reference

### When to Use Joins

- INNER JOIN → Only matching records
- LEFT JOIN → All from left table
- RIGHT JOIN → All from right table
- FULL JOIN → All from both tables

### Subquery Tips

- Single-row: Use `=`, `>`, `<`
- Multi-row: Use `IN`, `ANY`, `ALL`
- Correlated: Can be slow (executes per row)

### Window Function Tips

- ROW_NUMBER: Always unique, handle ties with next number
- RANK: Ties same rank, skip next rank
- DENSE_RANK: Ties same rank, continue ranking without skips
- PARTITION BY: Reset calculations per group

### Normalization Tips

- **1NF** → Eliminate repeating groups
- **2NF** → Eliminate partial dependencies
- **3NF** → Eliminate transitive dependencies
- **BCNF** → Every determinant is a superkey
- **Most apps:** 3NF provides good balance